CASE STUDY

An accounting practice its clients can actually see into

A working accountant used to run their practice on email attachments and folders on a desktop. This replaced that: clients submit requests, exchange documents and watch their own reporting periods progress, while the accountant runs everything from one admin panel.

ROLE	BACKEND	FRONTEND	STATUS
Sole developer	ASP.NET Core 8	React 19 · Vite	In real use

THE PROBLEM

Nobody could answer "where is my filing right now?"

Not the client, who had to ask. Not the accountant, who had to go and look. Every status question cost two people a context switch, and the answer lived in somebody's memory rather than in a system.

Documents arrived as attachments

Scattered across an inbox, with no record of which had been reviewed and which were still waiting.

Progress existed only as a habit

The same checklist ran every month, but it lived in the accountant's head, so it could not be shown to anyone.

Deadlines were tracked by memory

Filing dates are fixed and unforgiving, and nothing in the workflow surfaced the next one.

WHAT IT DOES

Three areas, one deployment

One React application and one ASP.NET Core API. What a person sees is decided by the roles in their token, not by which build they were given.

ANONYMOUS

Public site

The practice's shop window, and where new clients arrive.

- Services and pricing packages, admin-managed
- Rate-limited request form that emails the accountant
- Blog fed from an external RSS feed via a caching endpoint
- Testimonials, submitted by clients and published after review

AUTHENTICATED

Client portal

Each client sees their own data and nothing else.

- Dashboard with one concrete next step
- Document upload and download via pre-signed URLs
- Reporting periods with checklist progress
- Requests, subscriptions, password change

ADMIN ROLE

Accountant's panel

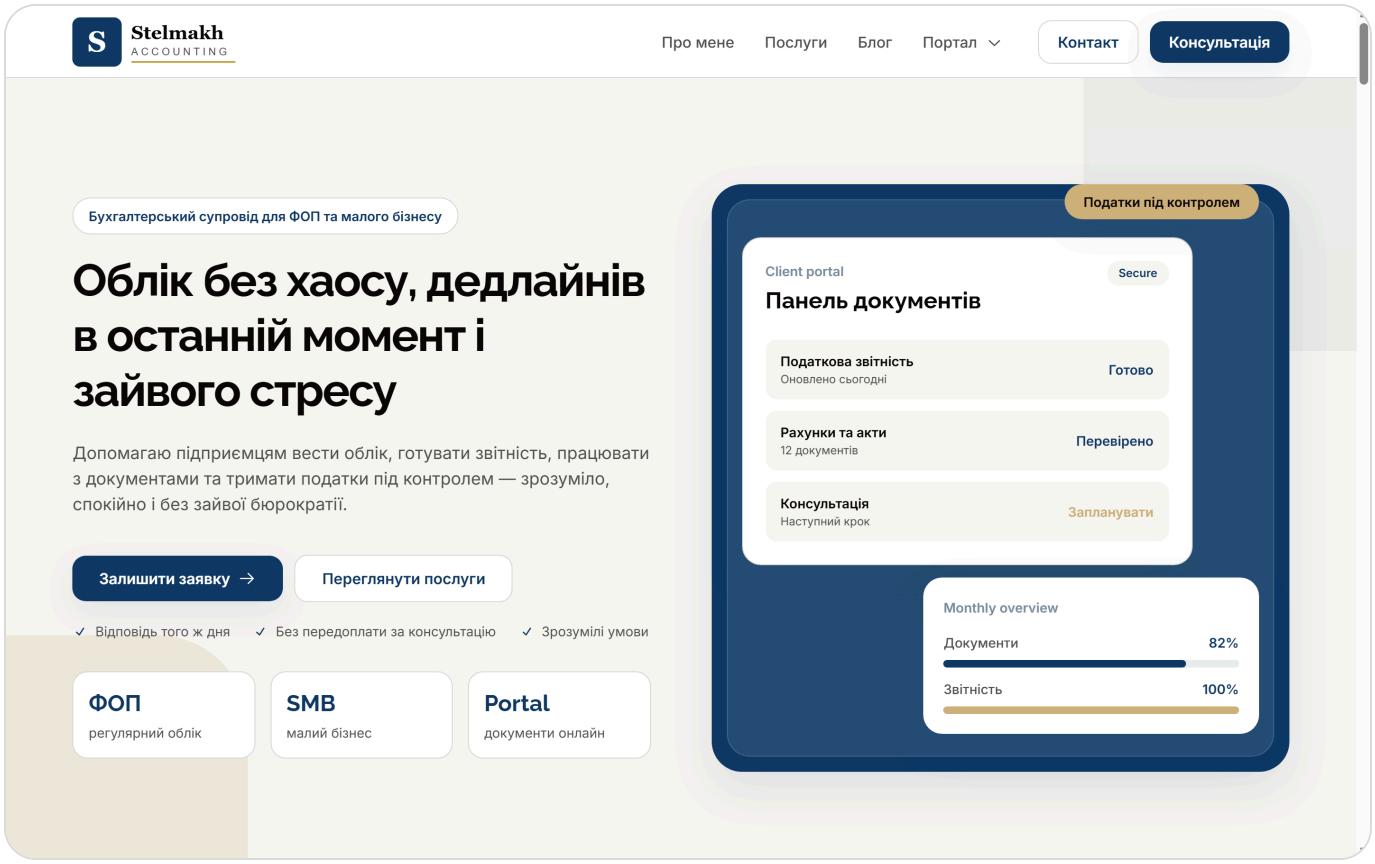
The working surface that drives everything above.

- Users: search, roles, access, password resets
- Requests, documents, subscriptions
- Checklist templates and reporting periods
- Services, packages, testimonial moderation

SCREENS

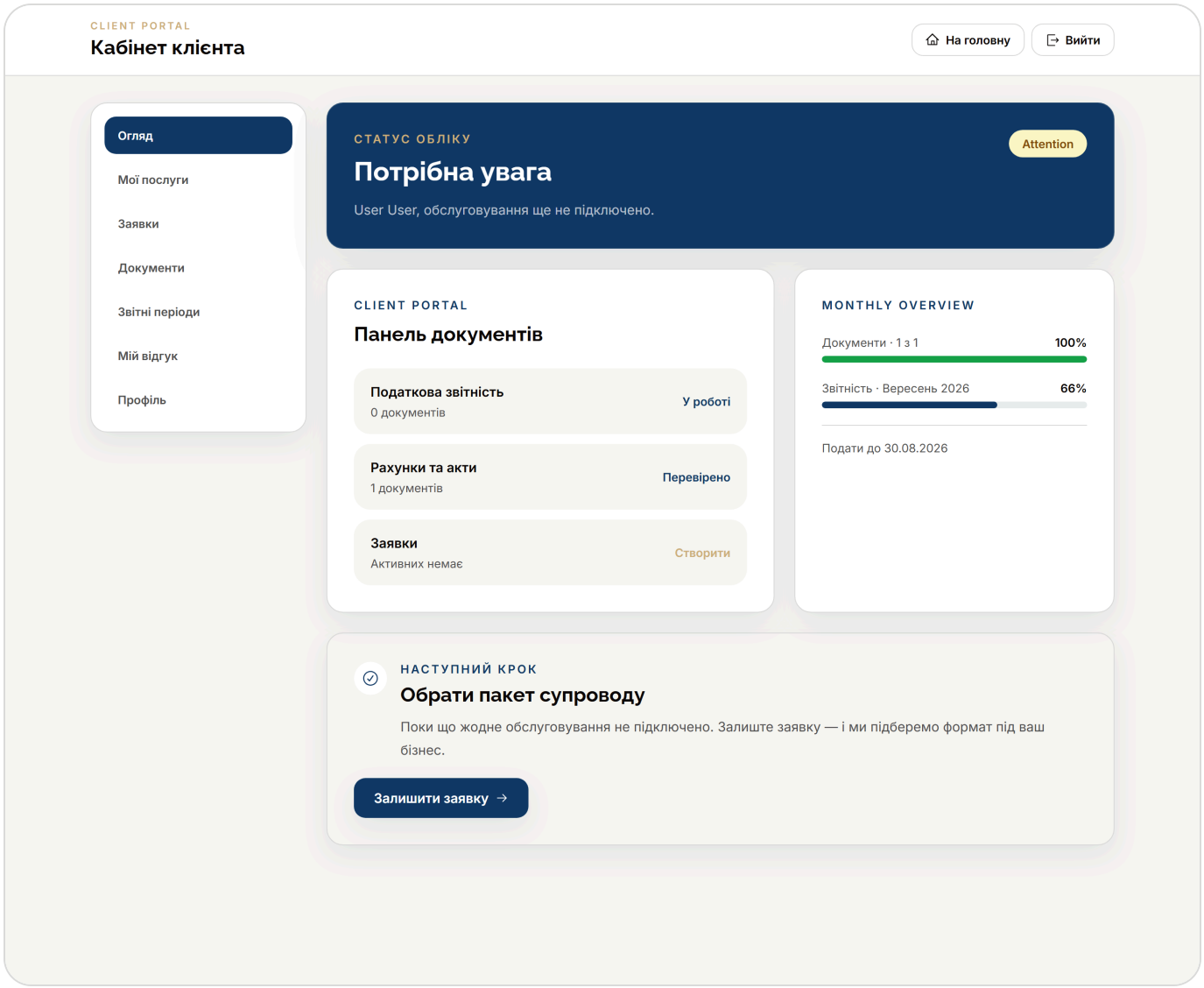
What it looks like

Screenshots from the running application. The interface is in Ukrainian because its users are — the accountant and their clients.



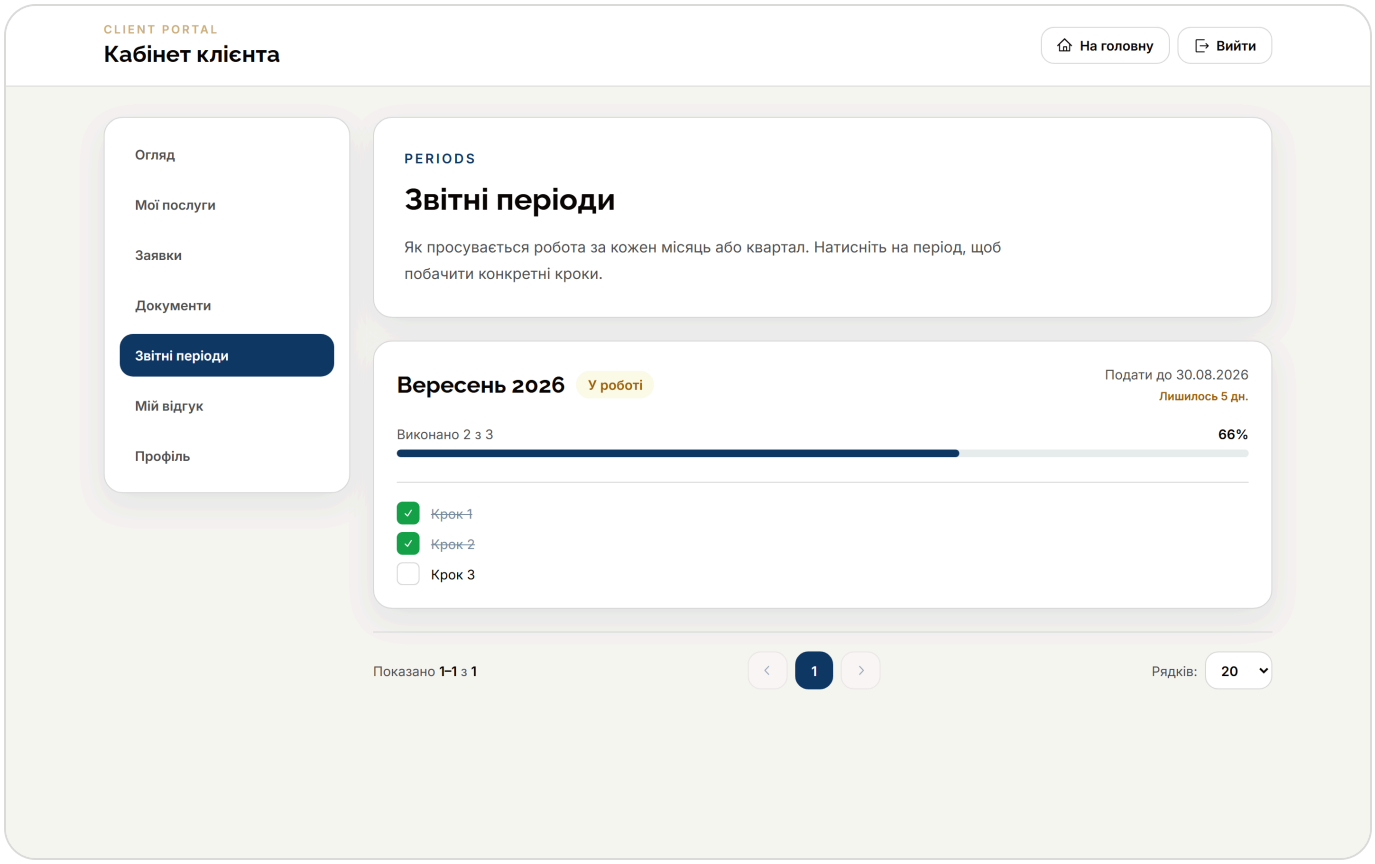
Public landing page

Services, packages and testimonials are all admin-managed content rather than markup, so the accountant changes them without a deploy.



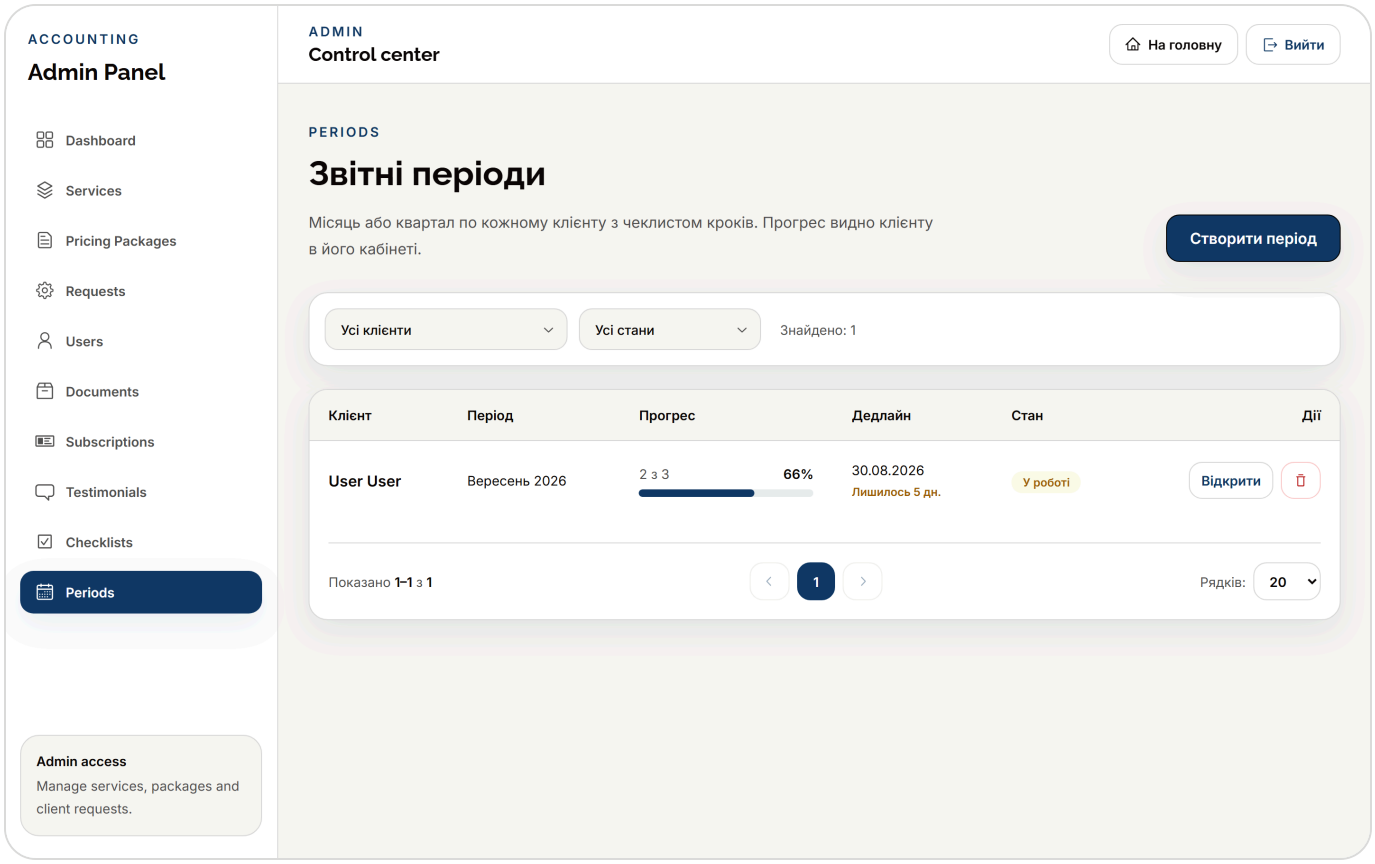
Client dashboard

The "next step" is computed in priority order — no active subscription outranks an overdue period, which outranks unprocessed documents. A dashboard offering five equal options tells the reader nothing.



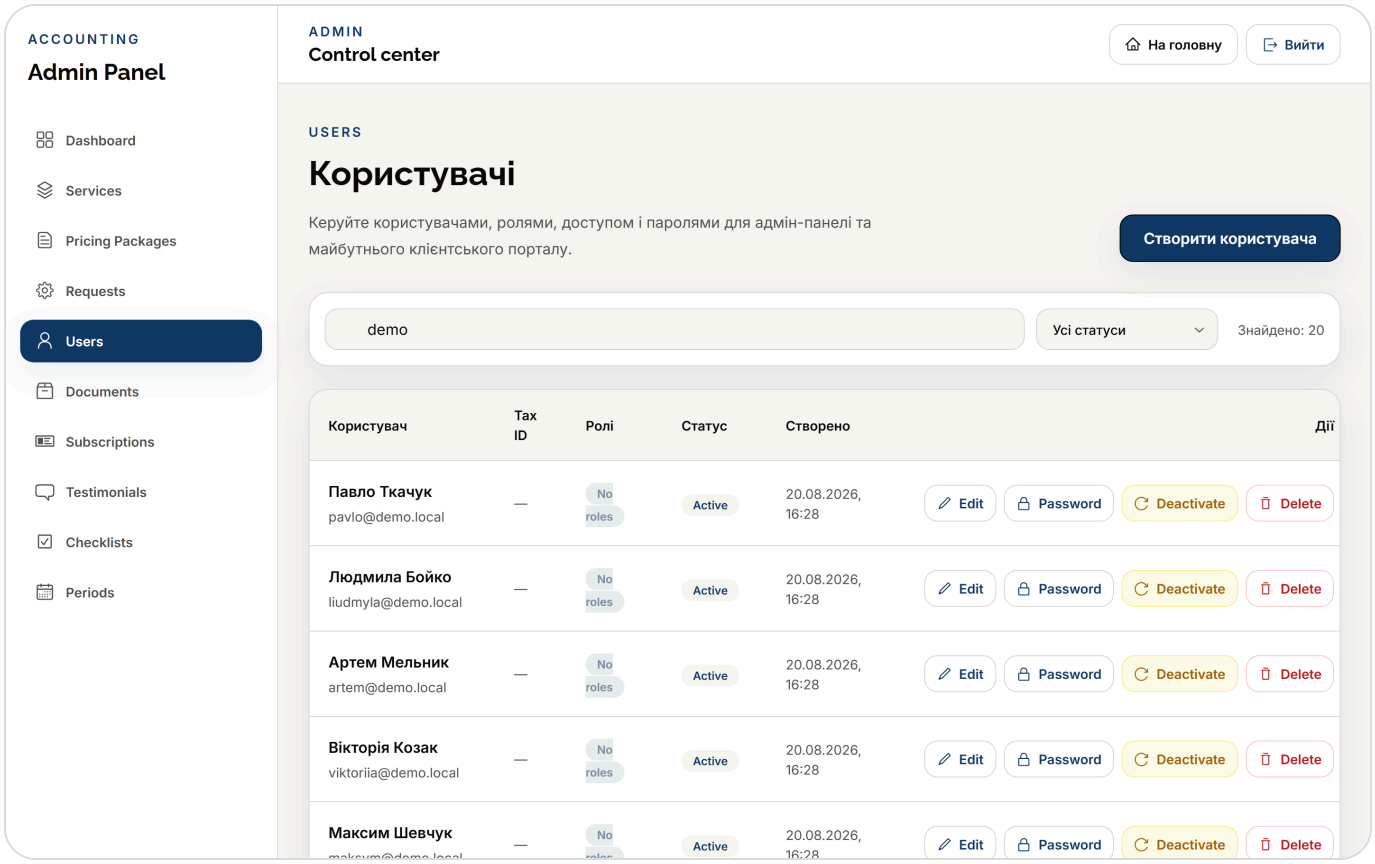
Reporting period, client side

The checklist was copied from a template when the period was created. The percentage is derived from the task list on read, never stored.



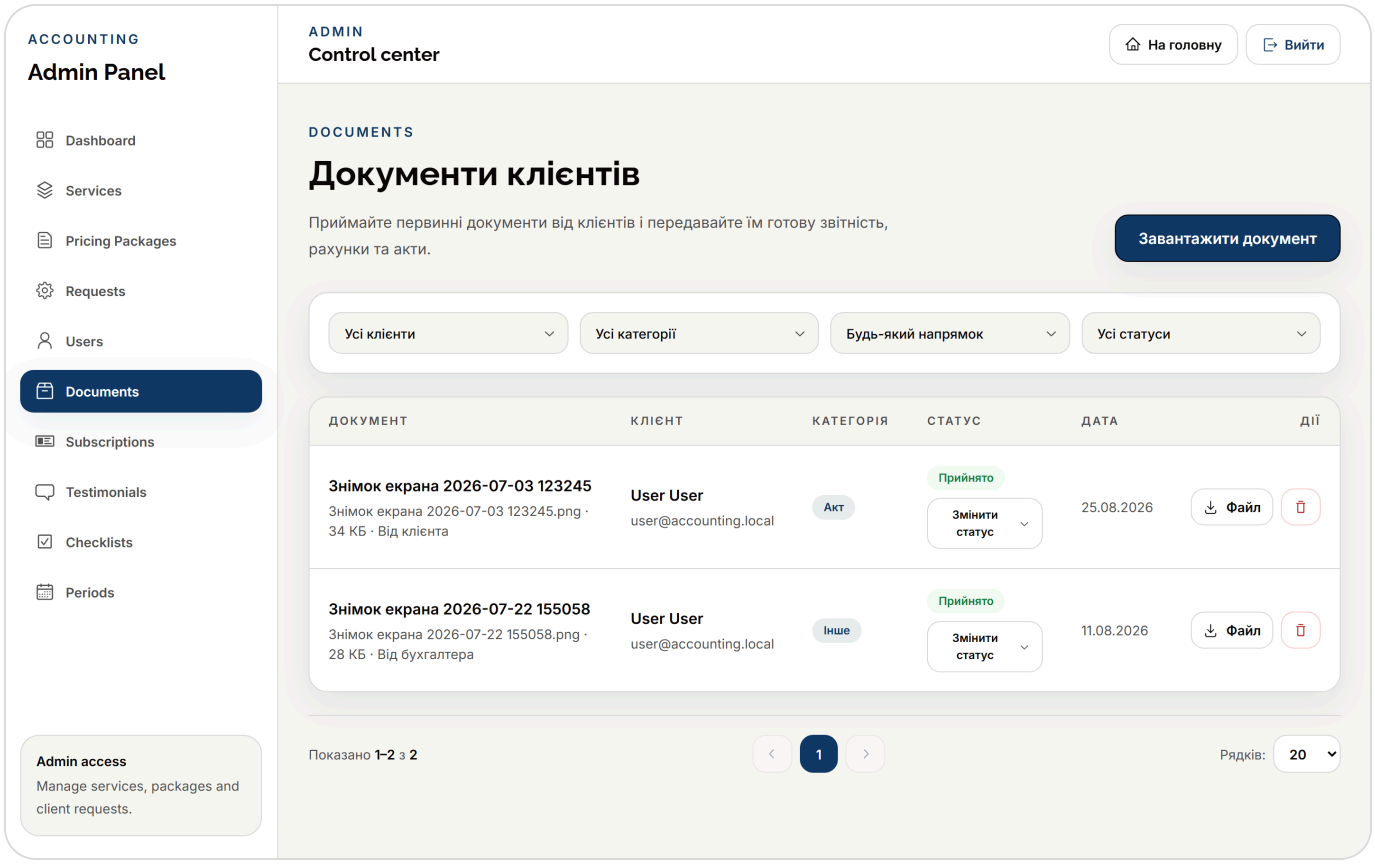
Reporting periods, admin side

Every client's period in one paginated list, with progress, deadline and status. One period per client per year/kind/number is enforced by a unique index.



User management

Server-side search across name, email and tax id using Postgres ILIKE, debounced on the client, paginated on the server.



Document exchange

Files live in Cloudflare R2. The API signs a short-lived URL and the browser talks to storage directly, so file bytes never pass through the application server.

ENGINEERING DECISIONS

The parts worth explaining

Every one of these was a choice with a cost on the other side. These are the trade-offs, not a feature list.

01

A reporting period copies its checklist instead of referencing it

Templates change when tax rules change. If a period pointed at its template, editing that template in November would retroactively add an unfinished step to an August period that was already filed. Periods store a snapshot, so a finished period stays a record of what was actually done at the time. The cost is duplicated rows; the benefit is that history cannot be rewritten by an unrelated edit.

02

Progress is computed, never stored

`ProgressPercent` is derived from the task list every time it is read, and EF Core is told to ignore it. A stored column would be a second source of truth about the same fact, and it starts lying the first time somebody ticks a task without recalculating. Deriving it costs nothing at this scale and cannot drift.

03

Aggregate roots enforced by the compiler, not by review

Child collections are private lists exposed as `IReadOnlyCollection<T>`, and the children's mutating methods are `internal`. A caller cannot tick a task without going through its period, which is where the "a closed period cannot be edited" rule lives. Invariants that depend on everyone remembering them are not invariants.

04

A unique index as the concurrency guard

One period per client per year, kind and number. Checking for an existing row and then inserting leaves a window two concurrent requests will walk straight through. The database constraint closes it, and the handler translates the violation into a clear message.

05

Pre-signed URLs rather than proxying file bytes

The API signs a short-lived URL and steps out of the transfer. A free-tier container never streams a 20 MB scan, and an expired link stops working on its own. Getting there meant discovering that R2 rejects the AWS SDK's default chunked signing — the fix was disabling chunk encoding and payload signing.

06

Checklist templates are data, not code

Hard-coding "group 3 files quarterly" would mean editing C# and redeploying every time the rules change — a cadence the code cannot keep up with. The accountant maintains templates from the admin panel instead, and each client gets a default one.

ARCHITECTURE

A layered monolith, on purpose

Four projects, one deployable process, one database. The project count is a compile-time boundary, not a service boundary — and at one accountant with tens of clients, distributing it would buy failure modes without buying a solution.

Accounting.Api

Controllers, JWT authentication, role policies, rate limiting on public endpoints, RFC 7807 error responses, DI composition.

Accounting.Application

MediatR commands and queries in feature folders, FluentValidation in the pipeline, DTOs, pagination primitives.

Accounting.Domain

Entities, aggregate roots, invariants. References no other project and no infrastructure package, which is what makes it testable in isolation.

Accounting.Infrastructure

EF Core with Npgsql, ASP.NET Core Identity, R2 storage, Brevo email — all implementing interfaces declared one layer up.

Dependencies point inwards

Infrastructure depends on Application, never the reverse. Swapping the storage provider touches one class and no use case.

Deployed as one container

Docker on Render, PostgreSQL on Neon, files on Cloudflare R2, frontend on Vercel.

TESTING

44 domain tests that are allowed to fail

The suite is deliberately domain-only. The entities have no infrastructure dependencies, so the tests need no database, no mocks and no fixtures — they run in about 40 ms, which is what makes them worth running on every change.

Rules, not properties

Every test targets something the entity promises to enforce. A test that would still pass with the rule deleted is not worth having.

Boundaries as theories

Period numbers are checked per periodicity with `[Theory]` — including quarter 12, a legal month number that is nonsense as a quarter and was a real bug in the UI.

Verified by mutation

The suite was checked by deliberately breaking domain rules and confirming the right test went red. A test that has never failed is decoration, not coverage.

KNOWN LIMITS

What I would do next

The system is in daily use, which makes its remaining gaps concrete rather than hypothetical.

Refresh tokens

Access tokens expire mid-session; the refresh flow is not built yet.

Route-level code splitting

The frontend ships as one ~1.4 MB chunk; admin code loads for anonymous visitors.

Cold starts

The free hosting tier sleeps, so the first request of the day is slow.

Integration tests

The domain is covered; the handlers and endpoints are not.